Noise-Aware Promotion for Self-Evolving GPU Kernels

A Statistical Extension to Cross-Run Kernel Optimization

Lee Si Yuan
Independent Researcher
siyuanlee.cs@gmail.com
November 2026

Abstract

Self-evolving kernel optimizers retain validated implementations as references for later runs. If promotion depends on a small set of noisy timings, a transient measurement advantage can become persistent system state. We study a fixed-sample promotion rule based on paired log-latency measurements and a minimum practical speedup. A candidate is promoted only when the upper one-sided confidence bound on its mean log-latency difference clears that speedup threshold. We evaluate the rule in a reproducible CPU Monte Carlo model with shared timing drift and independent Gaussian jitter. With 32 pairs and a 1% minimum gain, the simulated gate promoted a candidate with a true 5% speedup in 30,000 of 30,000 trials and a candidate with no speedup in 3 of 30,000 trials. These are model-specific simulation outcomes, not estimates of GPU error rates. The study motivates randomized, paired hardware evaluation before using such a gate in a kernel optimization system.

1. Introduction

Systems that search for improved GPU kernels need to decide when a candidate should replace a known-good implementation. This decision matters especially in a self-evolving workflow, where a promoted implementation can become the reference point for subsequent optimization. ForgeKernel describes such a workflow for kernel generation and optimization [1].

Kernel latency varies with device state, clocks, contention, launch conditions, and measurement overhead. Choosing the minimum observed latency therefore tends to favor candidates that benefited from favorable noise. This paper studies a statistical promotion rule that compares reference and candidate timings in nearby paired blocks. Taking logarithms makes a difference correspond to a multiplicative latency ratio.

Our contribution is limited to (1) a fixed-sample decision rule with an explicit practical-gain threshold and (2) a reproducible synthetic experiment comparing paired and unpaired

measurements under shared drift. We do not reproduce ForgeKernel, benchmark a GPU, or claim a measured improvement in kernel performance.

2. Promotion rule

For measurement block `i`, let `B_i > 0` and `C_i > 0` be the reference and candidate latencies. Define

```
b_i = log(B_i),   c_i = log(C_i),   d_i = c_i - b_i.
```

Negative `d_i` favors the candidate. Let $\bar{d}$ and `s_d` be the sample mean and sample standard deviation of the `n` paired differences. Given a minimum practical speedup `g > 0`, the candidate is promoted when

```
d̄ + z_0.95 * s_d / sqrt(n) < -log(1 + g),
```

where `z_0.95 = 1.6448536269`. The right-hand side is the log-latency difference corresponding to speedup `g`, since `C/B < 1/(1+g)` is equivalent to `log(C) - log(B) < -log(1+g)`. The left-hand side is a one-sided normal upper confidence bound for the mean difference. This approximation uses a normal critical value with an estimated standard error.

The rule is fixed-sample: it assumes the sample count is chosen before measurements and does not permit unadjusted repeated interim looks. If the rule does not pass, the existing reference is retained. A deployment policy also needs functional correctness, numerical tolerance, memory-safety checks, and regression checks for the target workload set. Screening many candidates requires an explicit multiple-comparison or sequential-testing policy.

Pairing helps only when the measurements share transient conditions. If a block has common additive log drift `U_i`, so that `b_i = μ_B + U_i + ε_B,i` and `c_i = μ_C + U_i + ε_C,i`, then subtraction yields `d_i = (μ_C - μ_B) + (ε_C,i - ε_B,i)` and cancels `U_i`. This algebra does not protect against order effects, candidate-specific drift, autocorrelation, or changes in workload. Hardware experiments should randomize or counterbalance execution order.

3. Experimental method

The standard-library Python experiment in `noise_aware_promotion_experiment.py` uses seed `20261105` and 30,000 trials per cell. Each paired observation has shared Gaussian log drift with standard deviation `0.05`; each timing also has independent Gaussian jitter with standard deviation `0.02`. The candidate's mean log latency is shifted by `-log(1+s)` for true speedup `s` in `{0, 0.5, 1, 2, 5}%`.

We compare the paired rule with an unpaired gate based on two independently sampled timing series. Both gates use the same one-sided normal critical value, a 1% minimum gain, and fixed sample counts `n` in `{8, 16, 32}`. In the unpaired condition, baseline and candidate series receive independent common-drift draws. Thus, this comparison isolates pairing under the model rather than describing every possible benchmarking protocol. Configuration and trial proportions are stored in `results.json`.

4. Results

Table 1 reports promotion proportions for selected cells. Each entry is based on 30,000 Monte Carlo trials.

Pairs	True speedup	Paired	Unpaired
8	0%	0.927%	0.003%
8	2%	29.253%	1.010%
8	5%	98.193%	39.923%
16	0%	0.157%	0.000%
16	2%	41.617%	0.787%
16	5%	99.987%	82.023%
32	0%	0.010%	0.000%
32	2%	62.470%	1.507%
32	5%	100.000%	99.853%

At 32 pairs, the paired rule promoted 3 of 30,000 zero-speedup candidates (0.01%) and all 30,000 candidates with a true 5% speedup. The finite simulation does not establish a zero false-promotion probability: with 30,000 trials, the observed proportion has limited resolution, and a zero count would not imply a zero underlying probability. At the 1% minimum-gain boundary, the promotion proportion is 5.39% for 32 pairs, consistent with the nominal one-sided boundary behavior in this model.

Under the null condition, the mean paired sample standard deviation was approximately `0.0281` log units at 32 pairs. The corresponding unpaired baseline and candidate values were approximately `0.0535`. This difference is expected because the paired construction removes the shared drift. The measured advantage is consequently conditional on the simulated covariance structure.

5. Implications and limitations

The rule provides a concrete evidence threshold for deciding whether a candidate replaces a retained reference. The simulation illustrates how paired measurements can improve sensitivity when they share substantial environmental drift. It does not establish that real GPU timings have this structure or that the stated threshold controls false promotions on a device.

The experiment uses synthetic Gaussian data, a normal critical value, and independent observations. It includes no GPU timings, kernel execution, thermal or clock transitions, autocorrelation, adaptive sample-size selection, multi-candidate search, correctness evaluation, or workload regressions. The shared drift is intentionally large relative to independent jitter. The reported rates describe this model and random seed only. A hardware study should use randomized paired blocks, prespecified stopping, calibrated uncertainty estimates, repeated null comparisons, and correction for the number of candidates tested.

6. Conclusion

We presented a fixed-sample, paired confidence-bound rule for kernel promotion and evaluated it in a transparent synthetic timing model. Pairing improved detection of larger gains under the model's shared-drift assumption. Hardware measurements remain necessary to calibrate the rule and assess its behavior in an actual GPU optimization workflow.

References

[1] Y. Chen et al., “ForgeKernel: Self-Evolving Kernels Catching and Surpassing Experts and Agents,” arXiv:2611.04998, 2026. https://arxiv.org/abs/2611.04998